

Python AI Bootcamp

Level 1: Python Foundations

Beginner Edition — Written for People Who Have Never Coded

Author: Venkatesan Vetrimurasu | YouTube: Venkat's Tech Lab

This guide assumes you have never written a line of code. We start from zero: what a computer program is, what a variable is, what a function is, and how to run Python. Then we walk through 17 small projects, explaining every line of code in plain English.

How to use this book: Install Python (see next page). Create a folder. For each topic, type the code yourself (do not only copy-paste). Run it. Read the line-by-line explanation. Then try the practice challenge. Breaking code and fixing it is the best way to learn.

Part 0 — Absolute Basics (Read This First)

What is a computer program?

A computer only does exactly what you tell it. A program is a list of instructions written in a language the computer understands. We write the instructions in a text file. When we "run" the file, the computer follows the instructions from top to bottom.

What is Python?

Python is one of those languages. It is popular because the code looks close to English, and it has free tools for almost everything (data, websites, AI, games). Almost all modern AI work is done in Python.

How to install Python (5 minutes)

1. Go to <https://www.python.org/downloads/> and download the latest version.
2. Run the installer. On Windows, tick the box "Add Python to PATH".
3. Open a terminal (Command Prompt on Windows, Terminal on Mac).
4. Type: `python --version` and press Enter. You should see a version number.
5. To run a file named `hello.py` type: `python hello.py`

Your first program

Create a file named `hello.py` and type only this one line:

```
print("Hello, world!")
```

Save the file. In the terminal, go to the folder that contains the file and run: `python hello.py`. The computer will print the words Hello, world! on the screen.

What just happened? `print` is a built-in command that shows text on the screen. The text inside the quotes is called a **string** (a piece of text). The parentheses `()` tell Python "run this command".

Variables — boxes with names

A variable is a name that stores a value. Think of it as a labeled box.

```
name = "Alice"      # the box named "name" now holds the text Alice
age = 25            # the box named "age" now holds the number 25
print(name)         # shows: Alice
print(age)          # shows: 25
print(name, age)    # shows: Alice 25
```

The equals sign `=` means "store the value on the right into the name on the left". It does NOT mean mathematical equality.

Types of values

- **int** — whole number: 10, -3
- **float** — decimal number: 3.14, 0.5
- **str** — text (string): "hello", 'AI'
- **bool** — True or False only

You can check a type with `type(value)`. Example: `type(25)` shows `int`.

f-strings — putting variables inside text

```
name = "Alice"
age = 25
print(f"My name is {name} and I am {age} years old.")
# Output: My name is Alice and I am 25 years old.
```

Put an `f` before the quotes, then write variable names inside curly braces `{}`. Python replaces them with the real values.

Comments

Anything after a `#` on a line is a comment. Python ignores it. Comments are notes for humans.

```
print("Hello")  # this is a comment, Python skips it
```

Indentation (very important)

Python uses spaces at the start of a line to show which lines belong together. Always use 4 spaces for one level of indentation. Wrong indentation causes errors.

Part 0 (continued) — Decisions, Loops, Functions

Making decisions with if

Sometimes you want the computer to do something only when a condition is true.

```
score = 85

if score >= 75:
    print("You passed!")
else:
    print("Try again.")
# Because 85 is greater than or equal to 75, it prints: You passed!
```

`if` checks a condition. The indented block under it runs only when the condition is `True`. `else` runs when the condition is `False`. Comparisons: `==` equal, `!=` not equal, `<` less, `>` greater, `<=` less or equal, `>=` greater or equal.

Loops — repeating work

A loop repeats a block of code. The most common is the `for` loop.

```
fruits = ["apple", "banana", "cherry"]

for fruit in fruits:
    print(fruit)
# Output:
# apple
# banana
# cherry
```

This means: take each item in the list one by one, call it `fruit`, and run the indented print line for that item.

Functions — reusable blocks

A function is a named block of code you can use many times. You define it once with `def`, then call it by writing its name with parentheses.

```
def greet(person):
    print(f"Hello, {person}!")

greet("Alice") # Output: Hello, Alice!
greet("Bob")   # Output: Hello, Bob!
```

`person` is a parameter — a temporary name for the value you pass in. When you write `greet("Alice")`, Python puts "Alice" into `person` and runs the body.

Lists — ordered collections

```
numbers = [10, 20, 30]
print(numbers[0]) # first item → 10 (counting starts at 0)
print(numbers[1]) # second item → 20
numbers.append(40) # add 40 at the end
print(numbers)    # [10, 20, 30, 40]
print(len(numbers)) # how many items → 4
```

Dictionaries — key to value maps

```
person = {"name": "Alice", "age": 25}
print(person["name"]) # Alice
print(person["age"])  # 25
person["city"] = "Chennai" # add a new key
print(person)
```

A dictionary stores pairs: a key and its value. You look up a value by its key inside square brackets.

You now know enough to understand the 17 projects. Each project starts with the idea in plain English, then shows the full code, then explains the code step by step.

The 17 Projects — Table of Contents

1. Variables, types, f-strings — Unit Converter
2. Lists, dicts, sets, tuples — Word Counter
3. Loops and comprehensions — Filter a Dataset
4. Functions, `*args`, `**kwargs`, lambdas — Calculator
5. File I/O + `pathlib` — Contact Manager
6. Error handling and custom exceptions
7. OOP and dataclasses — Dataset class
8. Modules, packages, `venv`, `pip` / `uv`
9. Reproducibility: seeds and pinned environments

10. Type hints with mypy
11. Pydantic: validation and settings
12. Generators, iterators, decorators
13. Context managers (with statement)
14. collections (defaultdict, deque, NamedTuple)
15. requests and REST APIs
16. Async with asyncio and httpx
17. Simple CLI with argparse

1. Variables, Types, f-strings — Unit Converter

What we are building

A small tool that converts a length from one unit to another (for example kilometers to miles).

New ideas in this project

- Storing related values in a dictionary (unit name → how many meters)
- Writing a function that takes inputs and returns a result
- Raising an error when the user asks for an unknown unit
- Formatting a number with a fixed number of decimal places

Full code

```
# 01_unit_converter.py
def convert_length(value, from_unit, to_unit):
    to_meters = {
        "m": 1.0,
        "km": 1000.0,
        "cm": 0.01,
        "mm": 0.001,
        "mile": 1609.34,
        "yard": 0.9144,
        "foot": 0.3048,
        "inch": 0.0254,
    }
    if from_unit not in to_meters or to_unit not in to_meters:
        raise ValueError(f"Unsupported unit. Supported: {list(to_meters.keys())}")
    meters = value * to_meters[from_unit]
    result = meters / to_meters[to_unit]
    return result

value = 5.0
from_u = "km"
to_u = "mile"
converted = convert_length(value, from_u, to_u)
print(f"{value} {from_u} = {converted:.4f} {to_u}")
# Output: 5.0 km = 3.1069 mile
```

Line-by-line explanation

1. `def convert_length(...):` — define a function named `convert_length` that accepts three inputs.
2. `to_meters = {...}` — a dictionary that says how many meters one unit equals.
3. `if from_unit not in to_meters...` — if the unit is unknown, stop and show an error message.
4. `meters = value * to_meters[from_unit]` — convert the input into meters.
5. `result = meters / to_meters[to_unit]` — convert meters into the target unit.
6. `return result` — give the answer back to whoever called the function.
7. We store 5.0 km, call the function, and print the answer with 4 decimal places (`:.4f`).

Practice: Add temperature ($^{\circ}\text{C} \leftrightarrow ^{\circ}\text{F}$) and weight ($\text{kg} \leftrightarrow \text{lb}$).

2. Lists, Dicts, Sets, Tuples — Word Counter

What we are building

A program that takes a paragraph of text, splits it into words, and counts how many times each word appears.

New ideas

- **list** — ordered collection you can change
- **dict** — key → value lookup table
- **set** — collection of unique items (no duplicates)
- **tuple** — ordered collection you cannot change
- **Counter** — a special dict that counts items for you

Full code

```
# 02_word_counter.py
from collections import Counter

text = """Python is great. Python is powerful.
Learning Python with hands-on projects is the best way."""

words = text.lower().replace(".", "").replace(",", "").split()
word_counts = Counter(words)
```

```
print("Word frequencies:")
for word, count in word_counts.most_common():
    print(f"{word:15} → {count}")

unique_words = set(words)
print(f"\nTotal unique words: {len(unique_words)}")
print(f"Unique words: {sorted(unique_words)}")
```

Line-by-line explanation

1. `from collections import Counter` — borrow the Counter tool from Python's collections library.
2. `text = """..."""` — store a multi-line string in the variable `text`.
3. `.lower()` makes everything lowercase so "Python" and "python" count as the same word.
4. `.replace(".", "")` removes periods. Same for commas.
5. `.split()` cuts the text into a list of words wherever there is a space.
6. `Counter(words)` counts how many times each word appears.
7. `most_common()` returns the words sorted from most frequent to least.
8. `set(words)` keeps only unique words. `len(...)` counts them.

Practice: Count pairs of consecutive words (bigrams).

3. Loops and Comprehensions — Filter a Dataset

What we are building

We have a list of student records. We want to keep only those who scored 75 or higher, build a dictionary of high scorers, and compute an average for older students.

New idea: list comprehension

A short way to build a new list from an old one in one line. `[s["name"] for s in students if s["score"] >= 75]` means: look at every student `s`; if the score is at least 75, put the name into the new list.

Full code

```
# 03_filter_dataset.py
students = [
    {"name": "Alice", "age": 22, "score": 88},
    {"name": "Bob", "age": 19, "score": 72},
    {"name": "Charlie", "age": 25, "score": 95},
    {"name": "Diana", "age": 20, "score": 61},
    {"name": "Eve", "age": 23, "score": 79},
]

passed = [s["name"] for s in students if s["score"] >= 75]
print("Passed:", passed)

high_scores = {s["name"]: s["score"] for s in students if s["score"] >= 80}
print("High scores:", high_scores)

older = [s for s in students if s["age"] > 21]
avg = sum(s["score"] for s in older) / len(older)
print(f"Average score (age > 21): {avg:.1f}")
```

Step-by-step

1. `students` is a list of dictionaries. Each dictionary is one student.
2. The first comprehension keeps names of students whose score is ≥ 75 .
3. The dict comprehension builds `{name: score}` only for scores ≥ 80 .
4. We filter students older than 21, sum their scores, and divide by how many there are.

Practice: Print the names of students who are both older than 20 and scored above 80.

4. Functions, *args, **kwargs, Lambdas — Calculator

What we are building

A flexible calculator that can add, multiply, or average any number of values, and can optionally print extra information.

New ideas

- `*args` — collect extra positional arguments into a tuple
- `**kwargs` — collect extra keyword arguments into a dict
- `lambda` — a tiny one-line function without a name

Full code

```
# 04_calculator.py
def calculate(operation, *args, **kwargs):
    if not args:
        raise ValueError("At least one number required")
    precision = kwargs.get("precision", 2)
    verbose = kwargs.get("verbose", False)

    if operation == "add":
        result = sum(args)
    elif operation == "multiply":
        result = 1
        for num in args:
            result *= num
    elif operation == "average":
        result = sum(args) / len(args)
    else:
        raise ValueError(f"Unknown operation: {operation}")

    result = round(result, precision)
    if verbose:
        print(f"Operation: {operation} | Numbers: {args} → Result: {result}")
    return result

square = lambda x: x ** 2
is_even = lambda x: x % 2 == 0

print(calculate("add", 10, 20, 30, precision=0, verbose=True))
print(calculate("multiply", 2, 3, 4, 5))
print(square(7))
print(list(filter(is_even, range(10))))
```

Step-by-step

1. `*args` gathers 10, 20, 30 into a tuple so we can accept any count of numbers.
2. `kwargs.get("precision", 2)` means: use the precision the user gave, or default to 2.
3. Depending on the operation string we add, multiply, or average.
4. `lambda x: x**2` is a tiny function that returns the square of x.
5. `filter(is_even, range(10))` keeps only the even numbers from 0 to 9.

Practice: Add a "power" operation. Try calling `calculate` with many numbers.

5. File I/O + pathlib — Contact Manager

What we are building

A tiny contact book that saves names, phones, and emails to a JSON file and can export them to CSV.

New ideas

- Reading and writing files with `open` and the `with` statement
- JSON — a text format for structured data (lists and dictionaries)
- CSV — a text format for tables (like a simple Excel sheet)
- `pathlib.Path` — modern way to work with file paths on any operating system

Full code

```
# 05_contact_manager.py
import json, csv
from pathlib import Path

CONTACTS_FILE = Path("contacts.json")

def load_contacts():
```

```
if CONTACTS_FILE.exists():
    with open(CONTACTS_FILE, "r", encoding="utf-8") as f:
        return json.load(f)
return []

def save_contacts(contacts):
    with open(CONTACTS_FILE, "w", encoding="utf-8") as f:
        json.dump(contacts, f, indent=2)

def add_contact(name, phone, email):
    contacts = load_contacts()
    contacts.append({"name": name, "phone": phone, "email": email})
    save_contacts(contacts)
    print(f"Added: {name}")

def export_to_csv(filename="contacts.csv"):
    contacts = load_contacts()
    with open(filename, "w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=["name", "phone", "email"])
        writer.writeheader()
        writer.writerows(contacts)
    print(f"Exported to {filename}")

def show_contacts():
    contacts = load_contacts()
    if not contacts:
        print("No contacts yet.")
        return
    for i, c in enumerate(contacts, 1):
        print(f"{i}. {c['name']} | {c['phone']} | {c['email']}")

add_contact("Alice", "123-456-7890", "alice@email.com")
add_contact("Bob", "987-654-3210", "bob@email.com")
show_contacts()
export_to_csv()
```

Step-by-step

1. `Path("contacts.json")` creates a path object for the file name.
2. `with open(...) as f` opens the file and guarantees it will be closed afterward.
3. `json.load(f)` reads the file and turns the JSON text into a Python list/dict.
4. `json.dump(...)` does the opposite: writes a Python object as JSON text.
5. `csv.DictWriter` writes rows of dictionaries as a CSV table.

Practice: Add a function that searches contacts by name.

6. Error Handling and Custom Exceptions

What we are building

A safe function that reads numbers from a file. If the file is missing, empty, or contains bad data, it explains the problem instead of crashing.

New ideas

- **try / except** — attempt risky code and catch problems
- **raise** — deliberately cause an error with a clear message
- Custom exception classes — your own error types for clearer messages

Full code

```
# 06_safe_reader.py
class FileEmptyError(Exception):
    pass

class InvalidFormatError(Exception):
    pass

def safe_read_numbers(filepath):
    try:
        with open(filepath, "r") as f:
            content = f.read().strip()
        if not content:
            raise FileEmptyError(f"File '{filepath}' is empty")
        numbers = []
        for line in content.splitlines():
            line = line.strip()
            if not line:
                continue
            try:
                numbers.append(float(line))
            except ValueError:
                raise InvalidFormatError(f"Invalid number found: '{line}'")
        return numbers
    except FileNotFoundError:
        print(f"Error: File '{filepath}' does not exist.")
        return []
    except (FileEmptyError, InvalidFormatError) as e:
        print(f"Custom Error: {e}")
        return []
    except Exception as e:
        print(f"Unexpected error: {e}")
        return []

with open("numbers.txt", "w") as f:
    f.write("10\n20\n3.5\nabc\n40\n")
print(safe_read_numbers("numbers.txt"))
print(safe_read_numbers("nonexistent.txt"))
```

Step-by-step

1. We create two new error types by inheriting from Exception.
2. Inside try we attempt to open and read the file.
3. If the file is empty we raise FileEmptyError with a clear message.
4. When converting a line to float fails, we raise InvalidFormatError.
5. Each except block catches one kind of problem and prints a friendly message instead of crashing.

Practice: Write a function that loads a JSON config and raises ConfigError if a required key is missing.

7. OOP and Dataclasses — Dataset Class

What we are building

A Dataset class that stores samples (features + optional label) and can print a short summary.

New ideas

- **Class** — a blueprint for objects that combine data and functions
- **Object / instance** — one concrete thing created from the class
- **Method** — a function that belongs to a class (uses self)
- **dataclass** — a shortcut that auto-creates the setup code for simple classes

Full code

```
# 07_dataset_class.py
from dataclasses import dataclass, field
from typing import List, Any
import statistics

@dataclass
class Sample:
    features: List[float]
    label: Any = None

@dataclass
class Dataset:
    name: str
    samples: List[Sample] = field(default_factory=list)

    def add_sample(self, features, label=None):
        self.samples.append(Sample(features, label))

    def __len__(self):
        return len(self.samples)

    def mean_feature(self, index):
        values = [s.features[index] for s in self.samples]
        return statistics.mean(values)

    def summary(self):
        print(f"Dataset: {self.name}")
        print(f"Number of samples: {len(self)}")
        if self.samples:
            n_features = len(self.samples[0].features)
            print(f"Features per sample: {n_features}")
            for i in range(n_features):
                print(f"    Feature {i} mean: {self.mean_feature(i):.2f}")

ds = Dataset(name="Iris-like")
ds.add_sample([5.1, 3.5, 1.4, 0.2], label="setosa")
ds.add_sample([7.0, 3.2, 4.7, 1.4], label="versicolor")
ds.add_sample([6.3, 3.3, 6.0, 2.5], label="virginica")
ds.summary()
print(f"Length: {len(ds)}")
```

Step-by-step

1. `@dataclass` tells Python to auto-generate the setup code for `Sample` and `Dataset`.
2. `self` means "this particular object". Every method receives `self` automatically.
3. `add_sample` creates a `Sample` and appends it to the list inside this `Dataset`.
4. `__len__` lets us write `len(ds)` instead of `len(ds.samples)`.
5. `summary` prints a readable overview of the dataset.

Practice: Add a method that returns the number of samples with a given label.

8. Modules, Packages, venv, pip / uv

A **module** is a Python file you can import. A **package** is a folder of modules. A **virtual environment** is an isolated Python setup for one project so libraries do not conflict. **pip** (or the faster tool **uv**) installs libraries.

```
# In the terminal:
python -m venv .venv
# Windows: .venv\Scripts\activate
# Mac/Linux: source .venv/bin/activate
pip install requests pandas
pip freeze > requirements.txt

# In Python you can then write:
# import requests
# import pandas as pd

def clamp(value, min_val, max_val):
    return max(min_val, min(value, max_val))

print(clamp(15, 0, 10)) # 10
print(clamp(-5, 0, 10)) # 0
```

***Why it matters:** Every AI project needs libraries. Virtual environments keep them organized and reproducible.*

9. Reproducibility — Seeds

Many programs use random numbers. If you do not control the randomness, every run gives different results and you cannot compare experiments fairly. A **seed** is a starting number that makes the random sequence the same every time.

```
# 09_reproducibility.py
import random
import numpy as np

def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)

set_seed(42)
print(random.random())
print(np.random.rand(3))
# Run the file twice – the numbers will be identical.
```

***Practice:** Call `set_seed(0)` and `set_seed(99)` and observe that the numbers change with the seed.*

10. Type Hints with mypy

Type hints are optional notes that say what kind of value a function expects and returns. They help you and other people read the code, and a tool called **mypy** can check them for mistakes.

```
# 10_type_hints.py
from typing import List, Dict, Optional

def average(numbers: List[float]) -> float:
    if not numbers:
        raise ValueError("List cannot be empty")
    return sum(numbers) / len(numbers)

def find_user(users: Dict[str, dict], user_id: str) -> Optional[dict]:
    return users.get(user_id)

print(average([10.0, 20.0, 30.0]))
# pip install mypy
# mypy 10_type_hints.py
```

`List[float]` means "a list of decimal numbers". `-> float` means "this function returns a decimal number". `Optional[dict]` means "a dictionary or None".

11. Pydantic — Validation

When data comes from a user or a file it may be wrong. Pydantic lets you describe the shape of the data as a class. It then automatically checks types, ranges, and formats, and gives clear error messages.

```
# 11_pydantic_demo.py
from pydantic import BaseModel, Field, EmailStr, field_validator

class User(BaseModel):
    id: int
```

```
name: str = Field(..., min_length=2, max_length=50)
email: EmailStr
age: int = Field(..., ge=0, le=120)

@field_validator("name")
@classmethod
def name_must_be_capitalized(cls, v):
    if not v[0].isupper():
        raise ValueError("Name must start with capital letter")
    return v

user = User(id=1, name="Alice", email="alice@example.com", age=28)
print(user.model_dump())
# pip install pydantic email-validator
```

If you pass age=150 or a bad email, Pydantic raises a `ValidationError` instead of letting bad data continue.

12. Generators, Iterators, Decorators

A normal function builds the whole result and returns it. A **generator** yields one value at a time and pauses, which saves memory for huge data. A **decorator** wraps a function to add extra behavior (for example timing) without changing the function itself.

```
# 12_generators_decorators.py
import time
from functools import wraps

def countdown(n):
    while n > 0:
        yield n
        n -= 1
print(list(countdown(5))) # [5, 4, 3, 2, 1]

def timer(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        end = time.perf_counter()
        print(f"{func.__name__} took {end - start:.4f} seconds")
        return result
    return wrapper

@timer
def slow_function():
    time.sleep(0.3)
    return "Done"
print(slow_function())
```

`yield` is like `return`, but the function can continue later. `@timer` means "wrap `slow_function` with the timer decorator".

13. Context Managers (with statement)

A context manager guarantees that setup and cleanup always happen. The classic example is opening a file: the file is closed automatically even if an error occurs. You write your own with a class that has `__enter__` and `__exit__`, or with the `@contextmanager` decorator.

```
# 13_context_managers.py
from contextlib import contextmanager
import time

class Timer:
    def __enter__(self):
        self.start = time.perf_counter()
        return self
    def __exit__(self, exc_type, exc_val, exc_tb):
        print(f"Elapsed: {time.perf_counter() - self.start:.4f}s")
        return False

with Timer():
    time.sleep(0.2)
    print("Working...")

@contextmanager
def open_and_count(path):
    f = open(path, "r", encoding="utf-8")
    try:
        lines = f.readlines()
        yield lines
    finally:
        f.close()
        print(f"Closed {path} ({len(lines)} lines)")

with open("demo_lines.txt", "w") as f:
    f.write("line one\nline two\n")
with open_and_count("demo_lines.txt") as lines:
    print([l.strip() for l in lines])
```

14. collections (defaultdict, deque, NamedTuple)

defaultdict creates a default value for missing keys so you do not need extra if-checks. **deque** is a list optimized for adding/removing from both ends (great for sliding windows). **NamedTuple** is a lightweight immutable record with named fields.

```
# 14_collections_demo.py
from collections import defaultdict, deque, namedtuple, Counter
```

```
grouped = defaultdict(list)
for cat, item in [("fruit", "apple"), ("fruit", "banana"), ("veg", "carrot")]:
    grouped[cat].append(item)
print(dict(grouped))

window = deque(maxlen=3)
for x in [10, 20, 30, 40, 50]:
    window.append(x)
    print(list(window))

Sample = namedtuple("Sample", ["features", "label"])
s = Sample(features=[1.2, 3.4], label="positive")
print(s.features, s.label)
print(Counter("abracadabra").most_common(3))
```

15. requests and REST APIs

A REST API lets programs talk over the internet. You send an HTTP request to a URL and receive a response (often JSON). The `requests` library makes this easy.

```
# 15_api_demo.py
import requests

def get_github_user(username):
    url = f"https://api.github.com/users/{username}"
    response = requests.get(url, timeout=10)
    if response.status_code == 200:
        return response.json()
    print(f"Error {response.status_code}")
    return {}

user = get_github_user("octocat")
if user:
    print(f"Name: {user.get('name')}")
    print(f"Public repos: {user.get('public_repos')}")
# pip install requests
```

Status code 200 means success. `response.json()` turns the JSON text into a Python dict.

16. Async with asyncio and httpx

Normal code waits for each network request to finish before starting the next. Async code can start many requests at once and wait for all of them together, which is much faster when you call many APIs or download many files.

```
# 16_async_demo.py
import asyncio
import httpx

async def fetch_url(client, url):
    response = await client.get(url, timeout=10)
    return f"{url} → {response.status_code}"

async def main():
    urls = [
        "https://httpbin.org/delay/1",
        "https://httpbin.org/delay/1",
        "https://api.github.com",
    ]
    async with httpx.AsyncClient() as client:
        tasks = [fetch_url(client, url) for url in urls]
        results = await asyncio.gather(*tasks)
    for r in results:
        print(r)

if __name__ == "__main__":
    asyncio.run(main())
# pip install httpx
```

`async def` defines an async function. `await` pauses until that step finishes. `asyncio.gather` runs many async tasks at the same time.

17. Simple CLI with argparse

A Command-Line Interface lets users pass options when they run a script, for example `python train.py --epochs 50 --lr 0.001`. The `argparse` module parses those options for you.

```
# 17_train_cli.py
import argparse
from pathlib import Path

def main():
    parser = argparse.ArgumentParser(description="Tiny training script")
    parser.add_argument("--data", type=Path, required=True, help="Path to dataset")
    parser.add_argument("--epochs", type=int, default=10, help="Number of epochs")
    parser.add_argument("--batch-size", type=int, default=32)
    parser.add_argument("--lr", type=float, default=1e-3, help="Learning rate")
    parser.add_argument("--verbose", action="store_true", help="Extra logs")
    args = parser.parse_args()

    print("=== Training Config ===")
    print(f>Data: {args.data}")
    print(f>Epochs: {args.epochs}")
    print(f>Batch size: {args.batch_size}")
    print(f>Learning rate: {args.lr}")
    if args.verbose:
        print("Verbose mode on")
    print("Done.")

if __name__ == "__main__":
    main()

# python 17_train_cli.py --data ./data --epochs 50 --verbose
# python 17_train_cli.py --help
```

Run the script with `--help` to see the automatic help message. This pattern appears in almost every real training script.

How to Practice from Zero

1. Install Python and confirm `python --version` works.
2. Create a folder for Level 1. Put each sample in its own `.py` file.
3. Type the code yourself (or copy carefully). Run it with `python filename.py`.
4. Read the line-by-line explanation. Change one thing and predict what will happen. Run it again.
5. Do the Practice challenge before moving to the next project.
6. When you finish all 17, build one small project that combines several ideas (for example a CLI contact manager that saves JSON).

Next Level

After Level 1 you are ready for **Level 2: Math and Data Tools** — NumPy, Pandas, Matplotlib, and the basic statistics and linear algebra used in machine learning.

Python AI Bootcamp • Level 1: Python Foundations (Beginner Edition)

Author: Venkatesan Vetrimurasu | YouTube: Venkat's Tech Lab

Start from zero. Type the code. Break it. Fix it. That is how you learn.